Backend Engineering Intern Case Study

Part 1: Code Review & Debugging

- ***Initial Implementation Provided:***

```
@app.route('/api/products', methods=['POST'])
def create_product():
    data = request.json

    # Create new product
    product = Product(
        name=data['name'],
        sku=data['sku'],
        price=data['price'],
        warehouse_id=data['warehouse_id']
    )

    db.session.add(product)
    db.session.commit()

    # Update inventory count
    inventory = Inventory(
        product_id=product.id,
        warehouse_id=data['warehouse_id'],
        quantity=data['initial_quantity']
    )

    db.session.add(inventory)
    db.session.commit()

    return {"message": "Product created", "product_id": product.id}
```

- ***Issues Identified:***

Issues	Problem	Impact
No transaction	Partial writes	Data inconsistency
SKU not unique	Duplicate SKUs	Wrong orders
Product tied to warehouse	Wrong schema	Not scalable
No validation	Crashes	Bad data

No concurrency handling	Race condition	Duplicate inserts

- ***Production-Ready Implementation:***

```
from flask import request, jsonify
from decimal import Decimal, InvalidOperation
from sqlalchemy.exc import IntegrityError
from datetime import datetime
import re

@app.route('/api/products', methods=['POST'])
@login_required
@limiter.limit("60 per minute")
def create_product():
    data = request.json or {}
    tenant_id = current_user.tenant_id
    user_id = current_user.id

    # ---------------------------
    # 1. Idempotency Check
    # ---------------------------
    idempotency_key = request.headers.get('Idempotency-Key')
    if idempotency_key:
        existing = IdempotencyRecord.query.filter_by(
            key=idempotency_key,
            tenant_id=tenant_id
        ).first()
        if existing:
            return jsonify(existing.response_body), 200

    # ---------------------------
    # 2. Input Validation
    # ---------------------------
    errors = []

    if not data.get('name') or len(data['name']) > 255:
        errors.append("Valid name is required (max 255 chars)")

    if not re.match(r'^[A-Za-z0-9\-_]{1,100}$', data.get('sku', '')):
        errors.append("Invalid SKU format")

    try:
        price = Decimal(str(data.get('price')))
        if price <= 0:
            errors.append("Price must be > 0")
    except:
```

```
    errors.append("Invalid price")

quantity = data.get('initial_quantity', 0)
if not isinstance(quantity, int) or quantity < 0:
    errors.append("Quantity must be non-negative integer")

if errors:
    return jsonify({"errors": errors}), 422

# ---------------------------
# 3. Transaction Block
# ---------------------------
try:
    with db.session.begin():

        # SKU uniqueness with lock
        existing = db.session.execute(
            db.select(Product)
            .where(
                Product.sku == data['sku'],
                Product.tenant_id == tenant_id
            )
            .with_for_update()
        ).scalar_one_or_none()

        if existing:
            return jsonify({"error": "SKU already exists"}), 409

        # ---------------------------
        # 4. Create Product
        # ---------------------------
        product = Product(
            tenant_id=tenant_id,
            name=data['name'].strip(),
            sku=data['sku'].strip().upper(),
            price=price,
            status=data.get('status', 'active'),
            expiry_date=data.get('expiry_date'),
            created_by=user_id,
            created_at=datetime.utcnow()
        )

        db.session.add(product)
        db.session.flush()

        # ---------------------------
        # 5. Create Inventory (Optional)
        # ---------------------------
```

```
        inventory_entries = []

        if data.get('warehouse_id') and quantity is not None:
            inventory = ProductInventory(
                product_id=product.id,
                warehouse_id=data['warehouse_id'],
                quantity=quantity,
                reserved_qty=0,
                batch_number=data.get('batch_number')
            )
            db.session.add(inventory)
            inventory_entries.append(inventory)

        # ---------------------------
        # 6. Audit Logging
        # ---------------------------
        audit_log = ProductAuditLog(
            product_id=product.id,
            action='created',
            changed_by=user_id,
            changed_at=datetime.utcnow(),
            old_values={},
            new_values={
                "name": product.name,
                "sku": product.sku,
                "price": str(product.price)
            }
        )
        db.session.add(audit_log)

    # ---------------------------
    # 7. Response Serialization
    # ---------------------------
    response_data = {
        "id": product.id,
        "name": product.name,
        "sku": product.sku,
        "price": str(product.price),
        "status": product.status,
        "expiry_date": product.expiry_date.isoformat() if product.expiry_date else
None,
        "inventory": [
            {
                "warehouse_id": inv.warehouse_id,
                "available_qty": inv.quantity - inv.reserved_qty,
                "batch_number": inv.batch_number
            }
            for inv in inventory_entries
```

```
            ],
            "created_at": product.created_at.isoformat()
        }

        # ---------------------------
        # 8. Save Idempotency Record
        # ---------------------------
        if idempotency_key:
            record = IdempotencyRecord(
                key=idempotency_key,
                tenant_id=tenant_id,
                response_body=response_data,
                created_at=datetime.utcnow()
            )
            db.session.add(record)
            db.session.commit()

        return jsonify(response_data), 201

    except IntegrityError:
        db.session.rollback()
        return jsonify({"error": "SKU conflict (concurrent request)"}), 409

    except Exception as e:
        db.session.rollback()
        return jsonify({"error": "Internal server error"}), 500
```

- ***Enhancements Added:***

1. Transaction handling (atomicity)
2. SKU uniqueness (DB + locking)
3. Multi-tenancy isolation
4. Input validation
5. Audit logging
6. Idempotency handling
7. Secure API response
8. Rate limiting

- ***Why Original Code Works But Fails in Production:***

The original code works in a controlled environment because it assumes:

- Single user access
- Valid inputs

- No concurrent requests

However, in production:

- Multiple users can create products simultaneously → race conditions
- Partial commits lead to inconsistent data
- Invalid inputs can crash the system
- Lack of tenant isolation can cause data leaks

Hence, while the code compiles and runs, it is not reliable, scalable, or secure for real-world usage.

- ***Edge Cases Considered:***

1. Concurrent product creation
2. Multi-warehouse inventory
3. Dead stock handling
4. Discontinued products
5. SKU conflicts
6. Expiry & FEFO

Part 2: Database Design

- ***Inventory Management System – Database Design:***

  The following schema is designed to support multi-warehouse inventory management, supplier relationships, and bundle products while ensuring scalability and data integrity.

```
-- ============================================================
-- CORE INVENTORY MANAGEMENT SCHEMA (TRIMMED VERSION)
-- ============================================================

-- ---------------------------
-- 1. TENANT (Multi-tenancy)
-- ---------------------------
CREATE TABLE tenants (
    id UUID PRIMARY KEY,
    name VARCHAR(255) NOT NULL
);

-- ---------------------------
-- 2. COMPANIES
-- ---------------------------
CREATE TABLE companies (
    id UUID PRIMARY KEY,
```

```
    tenant_id UUID NOT NULL REFERENCES tenants(id),
    name VARCHAR(255) NOT NULL
);

-- ---------------------------
-- 3. WAREHOUSES
-- ---------------------------
CREATE TABLE warehouses (
    id UUID PRIMARY KEY,
    tenant_id UUID NOT NULL REFERENCES tenants(id),
    company_id UUID NOT NULL REFERENCES companies(id),
    name VARCHAR(255) NOT NULL,
    location VARCHAR(255)
);

-- ---------------------------
-- 4. PRODUCTS
-- ---------------------------
CREATE TABLE products (
    id UUID PRIMARY KEY,
    tenant_id UUID NOT NULL REFERENCES tenants(id),
    name VARCHAR(255) NOT NULL,
    sku VARCHAR(100) NOT NULL,
    price DECIMAL(10,2) NOT NULL,
    product_type VARCHAR(50) DEFAULT 'simple', -- simple | bundle
    status VARCHAR(50) DEFAULT 'active',
    UNIQUE (tenant_id, sku)
);

-- ---------------------------
-- 5. INVENTORY (Product-Warehouse Mapping)
-- ---------------------------
CREATE TABLE product_inventory (
    id UUID PRIMARY KEY,
    tenant_id UUID NOT NULL REFERENCES tenants(id),
    product_id UUID NOT NULL REFERENCES products(id),
    warehouse_id UUID NOT NULL REFERENCES warehouses(id),
    quantity INT NOT NULL DEFAULT 0,
    reserved_qty INT NOT NULL DEFAULT 0,
    UNIQUE (product_id, warehouse_id)
);

-- ---------------------------
-- 6. INVENTORY TRANSACTIONS (History Tracking)
-- ---------------------------
CREATE TABLE inventory_transactions (
    id UUID PRIMARY KEY,
    tenant_id UUID NOT NULL REFERENCES tenants(id),
```

```
    product_id UUID NOT NULL REFERENCES products(id),
    warehouse_id UUID NOT NULL REFERENCES warehouses(id),
    change_quantity INT NOT NULL,  -- +ve or -ve
    transaction_type VARCHAR(50),  -- purchase | sale | adjustment
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- ---------------------------
-- 7. SUPPLIERS
-- ---------------------------
CREATE TABLE suppliers (
    id UUID PRIMARY KEY,
    tenant_id UUID NOT NULL REFERENCES tenants(id),
    name VARCHAR(255) NOT NULL
);

-- ---------------------------
-- 8. SUPPLIER-PRODUCT RELATION
-- ---------------------------
CREATE TABLE supplier_products (
    id UUID PRIMARY KEY,
    tenant_id UUID NOT NULL REFERENCES tenants(id),
    supplier_id UUID NOT NULL REFERENCES suppliers(id),
    product_id UUID NOT NULL REFERENCES products(id),
    cost_price DECIMAL(10,2),
    UNIQUE (supplier_id, product_id)
);

-- ---------------------------
-- 9. BUNDLE COMPONENTS (Self Relationship)
-- ---------------------------
CREATE TABLE bundle_components (
    id UUID PRIMARY KEY,
    tenant_id UUID NOT NULL REFERENCES tenants(id),
    bundle_product_id UUID NOT NULL REFERENCES products(id),
    component_product_id UUID NOT NULL REFERENCES products(id),
    quantity INT NOT NULL,
    UNIQUE (bundle_product_id, component_product_id)
);
```

- ***What is Fixed in This Schema :***

1. Separated **Product and Inventory** to support multiple warehouses
2. Introduced **product_inventory table** to track stock per warehouse
3. Added **inventory_transactions** to track all stock changes
4. Enforced **SKU uniqueness per tenant** to avoid duplication
5. Modeled **supplier-product relationship** using a many-to-many table

6. Implemented **bundle_components** to support composite products

- ***But:***

1. Should inventory allow negative stock?
2. Do we need batch or expiry tracking (FEFO)?
3. Can a product have multiple SKUs (barcode, vendor SKU)?
4. How are bundles priced?
5. Do suppliers have dynamic pricing over time?
6. Do we need reservation logic to prevent overselling?
7. Should warehouses support bin-level hierarchy?
8. Do we need audit logs for tracking changes?

To make this system production-ready, the following enhancements can be added:

- **Inventory reservation system** to prevent overselling
- **Batch/lot tracking with expiry dates** for FEFO handling
- **Audit logs** to track all changes for debugging and compliance
- **Idempotency and transaction handling** to avoid duplicate operations
- **Soft deletes** instead of hard deletes for data recovery
- **Indexing strategies** for faster queries on inventory and SKU lookup
- **Role-based access control (RBAC)** for multi-user environments

This design ensures a scalable and consistent inventory system while leaving flexibility for future extensions like order management and advanced warehouse operations.

- ***ER Diagram:***

```
TENANTS
  └── (1) ──────────────────────< (N) COMPANIES
                    │
                    └── (1) ──────────────────────< (N) WAREHOUSES
                                  │
                                  └── (1) ──────────────────────< (N)
PRODUCT_INVENTORY >──────────────────────── (N) PRODUCTS
                                                  │
                                                  └── (1) ──────────────────────<
(N) INVENTORY_TRANSACTIONS
```

PRODUCTS

├── (1) ──────────────────────< (N) PRODUCT_INVENTORY

├── (1) ──────────────────────< (N) INVENTORY_TRANSACTIONS

├── (N) ──────────────────────< (N) SUPPLIER_PRODUCTS
>────────────────────── (1) SUPPLIERS

└── (1) ──────────────────────< (N) BUNDLE_COMPONENTS
>────────────────────── (1) PRODUCTS (self-relation)

1. A Tenant can have multiple Companies
2. Each Company can have multiple Warehouses
3. A Product can exist in multiple warehouses via product_inventory
4. Inventory changes are tracked using inventory_transactions
5. A Supplier can provide multiple products and a product can have multiple suppliers (many-to-many)
6. A Product can be a bundle of other products using bundle_components (self-relation)

Part 3: API Implementation

- ***Low Stock Alert API – Implementation:***

```
@app.route("/api/companies/<uuid:company_id>/alerts/low-stock", methods=["GET"])

@login_required

def low_stock_alerts(company_id):



    tenant_id = current_user.tenant_id



    # Validate company

    company = db.session.execute(

        text("""

            SELECT id, name
```

```
        FROM companies
        WHERE id = :company_id
          AND tenant_id = :tenant_id
          AND deleted_at IS NULL
    """),
    {"company_id": str(company_id), "tenant_id": str(tenant_id)}
).mappings().first()

if not company:
    return jsonify({"error": "Company not found"}), 404

# Pagination
try:
    page = max(1, int(request.args.get("page", 1)))
    page_size = min(100, max(1, int(request.args.get("page_size", 20))))
except:
    return jsonify({"error": "Invalid pagination"}), 400

offset = (page - 1) * page_size

cutoff = datetime.utcnow() - timedelta(days=30)

query = text("""
    WITH sales AS (
        SELECT product_id, warehouse_id,
               ABS(SUM(quantity))/30.0 AS avg_daily_sales
```

```
    FROM inventory_transactions
    WHERE tenant_id = :tenant_id
      AND quantity < 0
      AND performed_at >= :cutoff
    GROUP BY product_id, warehouse_id
)

SELECT
    p.id AS product_id,
    p.name AS product_name,
    ps.sku,
    w.id AS warehouse_id,
    w.name AS warehouse_name,
    ist.qty_available AS current_stock,
    COALESCE(rr.reorder_point, 20) AS threshold,
    s.avg_daily_sales
FROM inventory_stock ist
JOIN products p ON p.id = ist.product_id
JOIN warehouses w ON w.id = ist.warehouse_id
LEFT JOIN reorder_rules rr ON rr.product_id = p.id AND rr.warehouse_id = w.id
LEFT JOIN sales s ON s.product_id = p.id AND s.warehouse_id = w.id
LEFT JOIN product_skus ps ON ps.product_id = p.id AND ps.is_primary = TRUE
WHERE ist.tenant_id = :tenant_id
  AND w.company_id = :company_id
  AND ist.qty_available <= COALESCE(rr.reorder_point, 20)
LIMIT :limit OFFSET :offset
```

```
""")

rows = db.session.execute(query, {
    "tenant_id": str(tenant_id),
    "company_id": str(company_id),
    "cutoff": cutoff,
    "limit": page_size,
    "offset": offset
}).mappings().all()

alerts = []

for r in rows:
    if r["avg_daily_sales"] and r["avg_daily_sales"] > 0:
        days = math.ceil(r["current_stock"] / r["avg_daily_sales"])
    else:
        days = None

    alerts.append({
        "product_id": str(r["product_id"]),
        "product_name": r["product_name"],
        "sku": r["sku"],
        "warehouse_id": str(r["warehouse_id"]),
        "warehouse_name": r["warehouse_name"],
        "current_stock": float(r["current_stock"]),
        "threshold": float(r["threshold"]),
```

```
        "days_until_stockout": days
    })

return jsonify({
    "alerts": alerts,
    "total_alerts": len(alerts),
    "page": page
}), 200
```

- ***Assumptions:***

1. Recent sales are calculated using the last **30 days of transactions**
2. Default reorder threshold is **20** if not specified
3. Stockout prediction is based on average daily sales
4. Alerts are generated per **product per warehouse**
5. Digital products are not considered for stock alerts

- ***Edge Cases Considered:***

1. Invalid company ID → returns 404
2. Pagination errors → returns 400
3. No recent sales → stockout days set to null
4. Missing reorder rules → default threshold used
5. Zero or negative stock handled correctly
6. Multi-tenant isolation enforced

- ***Approach:***

1. Used `inventory_stock` for current stock levels
2. Used `inventory_transactions` to calculate sales velocity
3. Joined warehouses to filter by company
4. Applied threshold using `COALESCE`
5. Implemented pagination for scalability

- ***Possible Improvements:***
- Add supplier details in response
- Include alert severity (critical, warning)
- Add caching for frequently accessed alerts

- Trigger notifications (email/webhooks)
- Add batch/expiry-based alerts (FEFO)

ER Diagram:

```
TENANTS
  └── (1) ──────────< (N) COMPANIES
                  │
                  └── (1) ──────────< (N) WAREHOUSES
                                  │
                                  └── (1) ──────────< (N) PRODUCT_INVENTORY
>────────── (N) PRODUCTS

PRODUCTS
  └── (1) ──────────< (N) PRODUCT_SKUS

PRODUCTS
  └── (1) ──────────< (N) BUNDLE_COMPONENTS >────────── (1) PRODUCTS (self
relation)

PRODUCTS
  └── (N) ──────────< (N) SUPPLIER_PRODUCTS >────────── (1) SUPPLIERS

PRODUCT_INVENTORY
  └── (1) ──────────< (N) INVENTORY_TRANSACTIONS
```

1. A **Tenant** can have multiple companies
2. Each **Company** can have multiple warehouses
3. A **Product** can exist in multiple warehouses via `product_inventory`
4. Inventory changes are tracked using `inventory_transactions`
5. Products can have multiple SKUs
6. Products can be bundled using a self-referencing relationship
7. Suppliers provide products through a many-to-many relationship